

Operating Systems Masterclass

Comprehensive Reference Guide & Architecture Breakdown

Presented by: Asad Usman

1. Introduction to Operating Systems

An **Operating System (OS)** is foundational system software that manages computer hardware and software resources, providing essential services for execution. It acts as an indispensable bridge between raw physical hardware and user applications.

Core Analogy: Just as a seasoned manager oversees every operational aspect of an enterprise, an operating system coordinates processors, storage, security, and peripheral units seamlessly.

Key Roles of an Operating System

- **Resource Allocator:** Manages CPU cycles, physical memory, secondary storage partitions, and I/O devices among multiple competing applications.
- **Control Program:** Supervises user program execution to prevent anomalies, unauthorized access, and system halts.
- **Hardware Abstraction:** Shields software developers from complex hardware register-level details by offering clean, uniform system calls and APIs.

Major System Components

- **Kernel:** The core control layer executing in privileged mode with absolute hardware access.
- **Shell / User Interface:** The interaction layer allowing users to issue commands via CLI or GUI.
- **System Calls:** Programming interfaces used by applications to request kernel services.

2. Processes and Threads

Execution in modern operating systems relies heavily on abstracting running code into manageable units of work: processes and threads.

Process Fundamentals

A **process** is an active program in execution containing program code, data, heap, and stack.

- **PCB (Process Control Block):** Data structure tracking state, program counter, and CPU registers.
- **Life Cycle States:** New, Ready, Running, Waiting, Terminated.

Threads (Lightweight Processes)

A **thread** is the fundamental unit of CPU utilization within a process.

- **Multithreading:** Enables concurrent execution paths within a single shared memory address space.
- **Concurrency vs Parallelism:** Time-interleaved execution versus simultaneous core execution.

3. CPU Scheduling

CPU scheduling determines which process in the ready queue is assigned to the processor next, aiming to maximize throughput, minimize turnaround time, and maintain fairness.

- **Preemptive vs. Non-Preemptive:** Preemptive schedulers can interrupt a running process (e.g., using time slices), whereas non-preemptive schedulers allow processes to run to completion or voluntary wait.
- **First-Come, First-Served (FCFS):** Simple queue execution; vulnerable to convoy effects.
- **Shortest Job First (SJF):** Prioritizes jobs with minimal execution duration.
- **Round Robin (RR):** Preemptive algorithm assigning fixed time quanta cyclically.
- **Priority Scheduling:** Allocates CPU based on specific priority numerical weights.

4. Memory Management

Memory management ensures secure, efficient allocation of primary memory (RAM) across multiple concurrent tasks without cross-process interference.

- **Logical vs. Physical Address:** CPU generates virtual logical addresses which are translated by hardware memory management units (MMU) into physical RAM locations.
- **Paging:** Eliminates contiguous physical requirements by dividing physical memory into *frames* and logical memory into *pages* mapped via Page Tables.
- **Virtual Memory:** Allows execution of processes exceeding physical RAM capacity via demand paging and secondary storage swap space.

5. File Systems and Storage

A file system structures, organizes, and retrieves persistent data stored on non-volatile media like HDDs and SSDs.

- **File Attributes:** Metadata tracking name, identifier, type, location, size, protection, and timestamps.
- **Directory Structures:** Hierarchical tree-based organization supporting absolute and relative file paths.
- **Allocation Strategies:** Contiguous allocation (fast, fragmentation-prone), Linked allocation (pointer-linked blocks), and Indexed allocation (centralized index blocks).

6. Operating System Families & Ecosystems

Different hardware form-factors and computing requirements have led to specialized operating system architectures worldwide.

Category	Operating System	Specialty & Typical Use Case
Desktop / Laptop	Microsoft Windows	Dominant consumer and enterprise desktop OS with unmatched gaming and software compatibility.
Desktop / Workstation	macOS	Proprietary UNIX-based Apple system optimized for creative media workflows and hardware integration.
Server / Cloud	Linux (Ubuntu, RHEL)	Open-source, highly secure, robust monolithic kernel powering global cloud infrastructure and servers.
Mobile	Android & iOS	Touchscreen-optimized portable OS architectures managing strict power, sensor, and app sandboxing constraints.
Embedded / RTOS	FreeRTOS, QNX	Real-time deterministic response systems utilized in automotive, medical, and industrial automation.

Summary Note: Understanding these core principles—from process scheduling and memory management to storage hierarchies—provides the foundational knowledge necessary for modern systems engineering and software development.